

PORTADA

Calidad de Software

Semana 14 — Patrones de diseño para pruebas automatizadas

Universidad de Antioquía · Ingeniería de Sistemas



REPASO RELÁMPAGO

Lo que ya tenemos de la semana 13

- Un **framework de automatización** (Selenium, Playwright o Cypress) que controla el navegador y ejecuta pasos por nosotros.
- Un **primer test E2E** que abre una página, llena campos y hace una aserción — con evidencia de ejecución.
- Una advertencia pendiente: ese script funciona, pero si el `id` de un campo cambia, hay que corregirlo en cada prueba que lo use.



APERTURA

Un solo cambio de diseño rompió 40 pruebas el mismo día

No cambió la lógica de negocio. Cambió el `id` de un botón. Las 40 pruebas tenían ese `id` copiado y pegado.

TRANSICIÓN

El problema no es automatizar, es cómo se organiza el código que automatiza

La semana 13 resolvió **con qué** se automatiza. Hoy resolvemos **cómo se estructura** esa automatización para que un cambio en la aplicación cueste corregir un solo lugar, no cuarenta.



CONCEPTO

Qué es un patrón de diseño de pruebas

Es una forma probada de **organizar el código de una suite automatizada** para que sea legible, no duplique conocimiento de la interfaz y sobreviva a los cambios normales de la aplicación. No cambia qué se prueba, cambia **cómo queda escrito** lo que se prueba.

Hoy vemos cuatro: **Page Object** y **Screenplay** (formas de organizar la interacción con la interfaz), y **fixtures** y **datos guiados por datos (data-driven)** (formas de organizar la preparación y la repetición de pruebas).



CONCEPTO

Page Object: una clase por pantalla

El patrón más usado en automatización de UI. Cada pantalla (o componente importante) de la aplicación se representa con una **clase** que expone dos cosas: los **localizadores** de sus elementos (dónde están el campo de usuario, el botón de enviar) y **métodos de alto nivel** para interactuar con ella (`login(usuario, clave)`).

El test deja de conocer `id` s o selectores CSS. Solo llama métodos de la clase. Si la pantalla cambia, se corrige **una sola clase**, no cada test que la usa.



ANTES

Sin Page Object: el selector, repetido en cada test

```
def test_login_exitoso(page):
    page.goto("https://the-internet.herokuapp.com/login")
    page.fill("#username", "tomsmith")
    page.fill("#password", "SuperSecretPassword!")
    page.click("button[type='submit']")
    assert "You logged into a secure area" in page.content()

def test_login_fallido(page):
    page.goto("https://the-internet.herokuapp.com/login")
    page.fill("#username", "usuario_malo")
    page.fill("#password", "clave_mala")
    page.click("button[type='submit']")
    assert "invalid" in page.content()
```

Dos tests, los mismos tres selectores copiados dos veces. Si `#username` cambia mañana, hay que tocar los dos — y todos los que vengan después.

DESPUÉS

Con Page Object: el selector vive en un solo lugar

```
class LoginPage:
    def __init__(self, page):
        self.page = page
        self.user = "#username"
        self.pwd = "#password"
        self.submit = "button[type='submit']"

    def login(self, usuario, clave):
        self.page.goto("https://the-internet.herokuapp.com/login")
        self.page.fill(self.user, usuario)
        self.page.fill(self.pwd, clave)
        self.page.click(self.submit)

def test_login_exitoso(page):
    LoginPage(page).login("tomsmith", "SuperSecretPassword!")
    assert "You logged into a secure area" in page.content()
```

El test ya no sabe qué es un `id`. Habla en el lenguaje del negocio: `login(usuario, clave)`.

CONCEPTO

Por qué esto mejora la mantenibilidad

Beneficio	Qué resuelve
Encapsulación	El selector vive en un solo archivo, no disperso en decenas de tests.
Fuente única de verdad	Un cambio de interfaz se corrige en un lugar y todos los tests quedan al día.
Legibilidad	El test se lee como una historia de usuario (<code>login</code> , <code>agregarAlCarrito</code>), no como manipulación de HTML.
Reutilización	Cualquier test que necesite iniciar sesión reutiliza la misma clase, sin copiar y pegar.



CONTRAEJEMPLO

Un Page Object mal hecho: aserciones dentro de la clase

```
class LoginPage:
    def login_y_verificar(self, usuario, clave):
        self.page.fill(self.user, usuario)
        self.page.fill(self.pwd, clave)
        self.page.click(self.submit)
        assert "secure area" in self.page.content() # NO va aquí
```

El Page Object describe **cómo interactuar** con la pantalla, no **qué se espera** del resultado. Meter `assert` dentro de la clase mezcla dos responsabilidades: si dos tests esperan resultados distintos del mismo flujo, uno de los dos queda mal servido. Las aserciones se quedan en el test; el Page Object solo actúa.

CONCEPTO

Screenplay: actores que ejecutan tareas

Alternativa a Page Object, pensada para suites grandes. En vez de una clase por pantalla, se modela la prueba como un **actor** (quien prueba) que tiene **habilidades** (ej. navegar en un navegador), ejecuta **tareas** (secuencias de pasos con sentido de negocio, como "iniciar sesión") y responde **preguntas** (consultas sobre el estado de la aplicación, usadas luego en la aserción).

La diferencia central con Page Object: Screenplay separa **quién actúa** de **qué pantalla usa**, lo que facilita reutilizar la misma tarea aunque cambie el canal (web hoy, móvil mañana).



EJEMPLO

El mismo login, en Screenplay

```
tomsmith = Actor.named("Tomsmith").whoCan(BrowseTheWeb.using(browser))

tomsmith.attemptsTo(
    Navigate.to("https://the-internet.herokuapp.com/login"),
    Enter.theValue("tomsmith").into(USERNAME_FIELD),
    Enter.theValue("SuperSecretPassword!").into(PASSWORD_FIELD),
    Click.on(SUBMIT_BUTTON),
)

assert tomsmith.asks_for(Text.of(BANNER)) == "You logged into a secure area"
```

La tarea `attemptsTo(...)` es la secuencia de negocio; la pregunta `asks_for(...)` es lo que se verifica. El actor es intercambiable — el mismo guion sirve para "Alice" o "Bob" con distintas credenciales.

TENSIÓN CONCEPTUAL

Page Object domina la industria, no Screenplay

Screenplay resuelve problemas reales que Page Object arrastra en suites grandes: pantallas que crecen hasta volverse clases de cientos de métodos, y dificultad para reutilizar pasos entre distintos canales. Sobre el papel, es el diseño más limpio.

Aun así, la gran mayoría de suites en producción usan Page Object. La razón no es técnica: Screenplay tiene **curva de aprendizaje más alta**, menos tutoriales y menos soporte en los frameworks populares. **La opción "mejor diseñada" no siempre gana si el equipo no la puede sostener.** Para un equipo de Fábrica-Escuela con una suite pequeña, Page Object es la opción razonable hoy.



CONCEPTO

Fixtures: preparar y limpiar sin repetir código

Una **fixture** es una función que prepara el estado que un test necesita antes de correr (abrir el navegador, iniciar sesión, cargar datos) y lo limpia después (cerrar sesión, cerrar el navegador) — sin que cada test tenga que escribir esos pasos.

Resuelve un problema distinto al de Page Object: no es **cómo** se interactúa con la pantalla, sino **qué debe existir** antes de que el test empiece a correr.



EJEMPLO

Una fixture de sesión iniciada, en pytest

```
import pytest

@pytest.fixture
def sesion_iniciada(page):
    LoginPage(page).login("tomsmith", "SuperSecretPassword!")
    yield page          # el test corre aquí
    page.click("a.button") # logout, al terminar

def test_ver_area_segura(sesion_iniciada):
    assert "Secure Area" in sesion_iniciada.content()
```

Cada test que reciba `sesion_iniciada` como parámetro arranca ya autenticado, sin repetir el flujo de login en su propio cuerpo.

CONCEPTO

Data-driven: un test, muchos datos

Pruebas **guiadas por datos (data-driven)**: en vez de escribir un test por cada combinación de entradas, se escribe **un solo test** y se le pasa una tabla de datos — cada fila se ejecuta como un caso independiente, con su propio resultado de aprobado o fallido.

Conecta directo con la semana 8: las particiones de equivalencia y los valores límite que ahí se diseñaron a mano son, en la práctica, las filas de esa tabla.



EJEMPLO

Un test, cuatro casos de login

```
import pytest

@pytest.mark.parametrize("usuario, clave, resultado_esperado", [
    ("tomsmith", "SuperSecretPassword!", "secure area"),
    ("tomsmith", "clave_mala", "invalid"),
    ("usuario_malo", "SuperSecretPassword!", "invalid"),
    ("", "", "invalid"),
])

def test_login(page, usuario, clave, resultado_esperado):
    LoginPage(page).login(usuario, clave)
    assert resultado_esperado in page.content()
```

Cuatro casos, cero duplicación de pasos. Agregar un quinto caso es agregar una fila, no un test nuevo.

INTERACCIÓN

Ejercicio rápido: ¿qué patrón usarían?

5 minutos, respondan en el chat:

1. La suite necesita probar el mismo formulario con 15 combinaciones distintas de campos válidos e inválidos.
2. Cada test de la suite necesita empezar con un carrito de compras ya cargado con dos productos.
3. Tres pantallas distintas comparten el mismo encabezado con el mismo botón de "cerrar sesión".

CONCEPTO EN DISPUTA

¿Page Object siempre vale el esfuerzo?

Es tentador presentar Page Object como una regla sin excepciones: "toda suite automatizada debe tener Page Objects". La realidad es más matizada. Para una suite de dos o tres pruebas que no va a crecer, envolver cada pantalla en una clase agrega una capa de indirección que cuesta más de escribir y mantener de lo que ahorra.

El patrón paga cuando la suite crece — cuando el mismo flujo se repite en varios tests o cuando la interfaz cambia con frecuencia. La pregunta correcta no es "¿aplico Page Object?", sino "¿esta suite va a vivir lo suficiente y crecer lo suficiente para que la inversión se pague?".



SÍNTESIS

Lo que hay que llevarse de hoy

1. **Page Object** mueve los selectores de la interfaz a una clase por pantalla — un cambio se corrige en un solo lugar.
2. Las **aserciones se quedan en el test**, nunca dentro del Page Object.
3. **Screenplay** modela actores, tareas y preguntas — más limpio en teoría, menos adoptado en la práctica por su curva de aprendizaje.
4. **Fixtures** preparan y limpian el estado que un test necesita, sin repetir esos pasos en cada uno.
5. **Data-driven** convierte un test en una tabla de casos — conecta con las particiones y valores límite de la semana 8.
6. Ningún patrón es gratis: **se paga cuando la suite crece**, no antes.



INSTRUCCIONES

Taller de hoy: refactorizar aplicando Page Object

2 horas, en sala de Zoom asignada — equipo fijo de Fábrica-Escuela

- Partan de la suite E2E que su equipo construyó en el taller de la semana 13 sobre **The Internet (Herokuapp)**.
- **Refactoricen** esa suite aplicando el patrón Page Object: extraigan una clase por cada pantalla involucrada, con sus localizadores y sus métodos de alto nivel. Los tests deben seguir corriendo en verde después del cambio.
- Las aserciones se quedan en el test — ningún `assert` dentro de la clase de Page Object.



INSTRUCCIONES

Entregable: comparativa antes/después

Además del código refactorizado, el equipo entrega un documento breve con:

- El **código antes** (semana 13) y el **código después** (con Page Object), uno junto al otro.
- Una tabla de **mantenibilidad**: líneas de código repetidas, cuántos archivos hay que tocar si el sistema cambia un selector, y cuánto costaría agregar un test nuevo del mismo flujo — antes vs. después.
- **Evidencia de ejecución** de la suite refactorizada corriendo en verde.

Formato: repositorio o carpeta compartida con el código de ambas versiones y un archivo markdown con la comparativa · **La entrega cierra al terminar las dos horas.** Recuerden: al enviar, cada integrante reporta en privado quién trabajó.



CIERRE

Para la próxima semana

Antes de la clase de la semana 15, hay que llegar con esto leído:

- Integración de la suite automatizada al pipeline de CI, y qué son las pruebas inestables (*flaky tests*).

Seguimos con los equipos fijos de **Fábrica-Escuela**. La semana 15 conecta la suite de hoy con el pipeline de la semana 12, para que corra en cada `commit`.

